

Received 30 March 2025; revised 29 June 2025; accepted 15 July 2025; date of publication 21 July 2025;
date of current version 22 August 2025.

Digital Object Identifier 10.1109/TQE.2025.3591213

Simulation of Shor Algorithm for Discrete Logarithm Problems With Comprehensive Pairs of Modulo p and Order q

KAITO KISHI¹ , JUNPEI YAMAGUCHI², TETSUYA IZU³,
AND NOBORU KUNIHIRO⁴ 

¹Quantum Laboratory, Fujitsu Research, Fujitsu Ltd., Kawasaki 211-8588, Japan

²Spatial Robotics Research Center, Fujitsu Research, Fujitsu Ltd., Kawasaki 211-8588, Japan

³Data and Security Laboratory, Fujitsu Research, Fujitsu Ltd., Kawasaki 211-8588, Japan

⁴Institute of Systems and Information Engineering, University of Tsukuba, Tsukuba 305-8573, Japan

Corresponding author: Kaito Kishi (e-mail: kishi.kaito@fujitsu.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TQE.2025.3591213>, provided by the authors.

ABSTRACT The discrete logarithm problem (DLP) over finite fields, commonly used in classical cryptography, has no known polynomial-time algorithm on classical computers. However, Shor has provided its polynomial-time algorithm on quantum computers. Nevertheless, there are only few examples simulating quantum circuits that operate on general pairs of modulo p and order q . In this article, we constructed such quantum circuits and solved DLPs for all 1860 possible pairs of p and q up to 32 qubits using a quantum simulator with PRIMEHPC FX700. From this, we obtained and verified values of the success probabilities, which had previously been heuristically analyzed by Ekerå (2019). As a result, the detailed waveform shape of the success probability of Shor's algorithm for solving the DLP, known as a periodic function of order q , was clarified. In addition, we generated 1015 quantum circuits for larger pairs of p and q , extrapolated the circuit sizes obtained, and compared them for $p = 2048$ bits between safe-prime groups and Schnorr groups. While in classical cryptography, the cipher strength of safe-prime groups and Schnorr groups is the same if p is equal, we quantitatively demonstrated how much the strength of the latter decreases to the bit length of p in the former when using Shor's quantum algorithm. In particular, it was experimentally and theoretically shown that when a basic adder is used in the addition circuit, the cryptographic strength of a Schnorr group with $p = 2048$ bits under Shor's algorithm is almost equivalent to that of a safe-prime group with $p = 1024$ bits.

INDEX TERMS Cryptography, digital signatures, public key cryptography, quantum circuit, quantum cryptography, quantum simulation.

I. INTRODUCTION

Let G be a finite multiplicative group, and let $\langle g \rangle$ denote the cyclic subgroup generated by an element g of G . The problem of finding an integer $0 \leq s < q$ such that $h = g^s$ for a given element $h \in \langle g \rangle$ is called the discrete logarithm problem (DLP). The difficulty of the DLP varies depending on the group, but when G is $\mathbb{F}_p^\times$, which is the multiplicative group of a finite field $\mathbb{F}_p$ of prime order p , no polynomial-time algorithm is known for classical computation. This difficulty is utilized to ensure the security of cryptographic algorithms, such as Diffie–Hellman (DH) key exchange [1] and digital signature algorithm (DSA) [2]. The number field sieve

(NFS) [3] is a known classical algorithm for solving the DLP in the multiplicative group $\mathbb{F}_p^\times$. Experimental results using the NFS have solved DLP instances with modulus p up to 240 digits (795 bits) (see Table 1), and the computational complexity of the NFS is subexponential in the size of p , specifically $O(\exp(((64/9)^{1/3} + o(1))(\ln p)^{1/3}(\ln \ln p)^{2/3}))$. Therefore, it is recommended to set p to be at least 2048 bits for DH key exchange and DSA [4]. According to the Pohlig–Hellman algorithm [5], a classical algorithm for solving the DLP in general cyclic groups, it is desirable for the order q to be a large prime. In the multiplicative group $\mathbb{F}_p^\times$, q always divides $p - 1$, so when $q = (p - 1)/2$ is prime (i.e.,

TABLE 1. Progress of Solving Records for the DLP in the Multiplicative Group of a Finite Field $\mathbb{F}_p^\times$

Date solved	Modulo	Solver	Remarks
18 June 2005	130 digits	Joux and Lercier [30]	strong-prime
22 December 2006	135 digits	Dorofeev et al. [31]	strong-prime
5 February 2007	160 digits	Kleijnung [32]	safe-prime
11 June 2014	180 digits	Bouvier et al. [33]	safe-prime
16 June 2016	232 digits	Kleijnung et al. [34]	safe-prime
2 February 2019	240 digits	Boudot et al. [35], [36], [37]	safe-prime

A strong-prime refers to a large prime number p for which the largest prime factors of both $p-1$ and $p+1$ are also large; for more details, see [29].

p is a safe-prime), the group has the highest resistance to the Pohlig–Hellman algorithm. In this article, we refer to pairs (p, q) where p and q are primes and $q = (p-1)/2$ as safe-prime groups. For DSA signatures, it is desirable for q to be small because modular arithmetic with the prime q is required. The computational complexity of square root algorithms such as the baby-step giant-step method [6], a classical algorithm for solving the DLP in general cyclic groups of order q , is exponential in the size of q , specifically $O(\sqrt{q})$. By comparing the computational time required for the NFS and square root algorithms, it is recommended to set q to 224 or 256 bits when p is 2048 bits, and q to 256 bits when p is 3072 bits [2], [7]. In this article, we refer to pairs (p, q) that satisfy such relationships as Schnorr groups. Importantly, in classical computation, if p matches and q is at least q of the Schnorr group, the computational effort required to solve the DLP is the same.

In 1994, Shor proposed a quantum algorithm that can solve the DLP in polynomial time [8], [9]. This algorithm does not tolerate errors in qubits, so it cannot solve large-scale problems on currently developed noisy intermediate-scale quantum computers. However, if a fault-tolerant quantum computer (FTQC) is realized, there would be a high risk that cryptographic algorithms based on the DLP will be broken.

Shor’s quantum algorithm can also solve the factorization problem in polynomial time, and estimation of the resources required to factor a 2048-bit Rivest–Shamir–Adleman (RSA) composite number has been reported [10], [11], [12]. In [10], the coset representation of modular integers, windowed arithmetic, and oblivious carry runways are used to minimize the number of Toffoli gates, and considerations are made for implementation using surface codes that account for the time of the magic state factory, which is state preparation necessary for applying Toffoli gates. This approach estimates the number of Toffoli gates and the computation time required to factorize a 2048-bit RSA composite number. Similarly, Gouzien and Sangouard [11] estimate the factorization time using fewer qubits, although with increased time compared to [10], by using 3-D color codes and quantum memory. Nevertheless, this resource estimate is highly dependent on the configuration of the actual quantum machine, and particularly, the implementation of non-Clifford gates has been significantly improved by methods such as space-time efficient

analog rotation quantum computing architecture (STAR architecture) [13], [14], [15], zero-level distillation [16], and magic state cultivation [17]. On the other hand, a bottom-up approach involves constructing quantum circuits that can be practically executed on simulators and extrapolating from the obtained gate counts to estimate the gate count for a 2048-bit composite number [12], [18].

There is less analysis on the estimation of solving the DLP using Shor’s algorithm compared to factorization. In [10], the computation time for DLP is estimated for both safe-prime groups and Schnorr groups. It shows that the latter can be solved in a shorter time than the former using Shor’s algorithm [8], Ekerå’s algorithm [19], and Ekerå–Håstad’s algorithm [20]. Furthermore, Hhan et al. [21] theoretically demonstrate that $\Omega(\log q)$ depth of group operation queries is required for solving the DLP. However, it remains unclear how p and q specifically affect the time complexity when including the most time-consuming part of Shor’s algorithm, the modular exponentiation. Although heuristic theoretical analysis of the success probability has been conducted by Ekerå [22], it needs to be verified whether this holds true when Shor’s quantum algorithm is executed precisely. As for implementation examples, there are experiments on simulators using Qiskit [23] and on actual quantum computers [24]. The former plots the number of gates and success probability, but does not consider how these depend on the pair of p and q . For the latter, only the experiment solving $2^x \equiv 1 \pmod{3}$ ($p=3, q=2$) has been successful, making it difficult to analyze for larger values. Especially, as quantum computers approach FTQC, Shor’s algorithm will likely be executed for larger p and q , so it is necessary to gather specific computational examples around this in advance. On the other hand, there is research extending Regev’s multidimensional version of Shor’s algorithm [25] to the DLP [26], but this will not be addressed here as it increases the number of qubits, making quantum simulation difficult despite the reduction in gate count.

In this study, we comprehensively simulated the quantum circuits of Shor’s algorithm for solving the DLP. We solved DLPs on a quantum simulator for all 1860 pairs of p and q possible up to 32 qubits using the Q-ADD [27] adder circuit and obtained the actual success probabilities. From this, we compared the distribution of success probabilities and the changes due to the size of the order q with Ekerå’s heuristic analysis [22]. In addition, we selected 1015 pairs of larger p and q and obtained the number of gates and depth of the quantum circuits before and after optimization using the R-ADD [28] adder circuit. Using these results, we extrapolated the circuit scale for 2048 bit, as in [12] and [18], in a bottom-up manner. In this way, from the perspective of computational complexity analysis, we specifically demonstrated how the cipher strength of the Schnorr group, which is equivalent to the safe-prime group in classical computation, decreases to the strength of a safe-prime group with a smaller p under quantum computation. Furthermore, we

demonstrated through theoretical analysis that this holds true regardless of the parameters used in the extrapolation formula.

In this article, we use the following notations. For a natural number $x > 0$, the logarithm to the base 2 is denoted by $\log x$, and the natural logarithm is denoted by $\ln x$. For a nonnegative integer x , its size (bit length) is denoted by $|x|$, and the function that rounds x to the nearest integer is denoted by $\lceil x \rceil$.

II. SHOR'S ALGORITHM FOR THE DLP

Shor's algorithm for solving the DLP can be divided into two main stages:

- 1) the quantum computation part, which involves the calculation of modular exponentiation, the inverse quantum Fourier transform, and measurement;
- 2) the classical computation part, which uses the measurement results to obtain s .

Each step will be described in sequence.

A. QUANTUM COMPUTATION PART

A set of qubits is divided into three registers, with the number of qubits in each register denoted as $v_1, v_2, w \in \mathbb{N}$, and the following state is prepared using the Hadamard gate:

$$\frac{1}{\sqrt{2^{v_1+v_2}}} \sum_{x_1=0}^{2^{v_1}-1} \sum_{x_2=0}^{2^{v_2}-1} |x_1\rangle |x_2\rangle |0^w\rangle. \quad (1)$$

Next, the X gate is applied to the least significant bit of the third register

$$\frac{1}{\sqrt{2^{v_1+v_2}}} \sum_{x_1=0}^{2^{v_1}-1} \sum_{x_2=0}^{2^{v_2}-1} |x_1\rangle |x_2\rangle |0 \cdots 01\rangle. \quad (2)$$

Then, the modular exponentiation $h^{x_1} g^{x_2} \bmod p$ is computed and stored in the third register. The specific method for this computation is described in Section III. The result of this computation is as follows:

$$\frac{1}{\sqrt{2^{v_1+v_2}}} \sum_{x_1=0}^{2^{v_1}-1} \sum_{x_2=0}^{2^{v_2}-1} |x_1\rangle |x_2\rangle |h^{x_1} g^{x_2} \bmod p\rangle. \quad (3)$$

This is equivalent to the following [38]:

$$\frac{1}{\sqrt{2^{v_1+v_2}q}} \sum_{l=0}^{q-1} \sum_{x_1=0}^{2^{v_1}-1} \sum_{x_2=0}^{2^{v_2}-1} e^{2\pi i(slx_1+lx_2)/q} |x_1\rangle |x_2\rangle |\hat{f}(sl, l)\rangle \quad (4)$$

$$= \frac{1}{\sqrt{2^{v_1+v_2}q}} \sum_{l=0}^{q-1} \left(\sum_{x_1=0}^{2^{v_1}-1} e^{2\pi i x_1 (sl/q)} |x_1\rangle \right) \cdot \left(\sum_{x_2=0}^{2^{v_2}-1} e^{2\pi i x_2 (l/q)} |x_2\rangle \right) |\hat{f}(sl, l)\rangle \quad (5)$$

where $|\hat{f}(l_1, l_2)\rangle = \frac{1}{\sqrt{q}} \sum_{j=0}^{q-1} e^{-2\pi i l_2 j/q} |g^j \bmod p\rangle$. Applying the inverse quantum Fourier transform results in the following:

$$\frac{1}{\sqrt{q}} \sum_{l=0}^{q-1} |\widetilde{sl/q}\rangle |\widetilde{l/q}\rangle |\hat{f}(sl, l)\rangle. \quad (6)$$

Here, $\widetilde{sl/q}$ in the first register is the v_1 -bit approximation of β/q in

$$sl/q = \alpha + \beta/q \quad (\alpha, \beta \in \mathbb{Z}, 0 < \beta < q) \quad (7)$$

and $\widetilde{l/q}$ in the second register is the v_2 -bit approximation of l/q . Measuring the first and second registers concludes the quantum computation part. The procedure up to this point is described by the quantum circuit shown in Fig. 1.

In [22], it is stated that the qubits of the first and second registers can be recycled using the semiclassical quantum Fourier transform with a uniform superposition. Since the quantum circuit is not shown, it is explicitly illustrated here as Fig. 2.

While the existing method in Fig. 2 can minimize the number of qubits in the control register to one, increasing it to two as in Fig. 3 allows the parts enclosed by the dashed lines to be executed simultaneously, reducing the time depth of the quantum circuit. The optimality of this is proved in the Appendix, available online. In addition, a faster method is shown in the Appendix for cases where the measurement time to obtain $m_0, \dots, m'_0, \dots$ is longer than each part of modular exponentiation. In the implementation on the quantum simulator [39], [40] used in this study, these methods utilizing the semiclassical inverse quantum Fourier transform were not employed due to the constraints of the simulator. However, in actual quantum computer implementations, these methods can be applied not only to the DLP but also to Shor's algorithm for integer factorization in general.

B. CLASSICAL COMPUTATION PART

The purpose of this part is to determine s from $\widetilde{sl/q}$ and $\widetilde{l/q}$ obtained in the quantum computation part. First, we determine $l \in \mathbb{Z}$ from $\widetilde{l/q}$. If $2^{v_2} > q$, l can be obtained by $l = \lceil \widetilde{l/q} \cdot q \rceil$. Similarly, if $2^{v_1} > q$, β can be determined from $\beta = \lceil \widetilde{sl/q} \cdot q \rceil$.

Since l and β have been determined up to this point, combining them with s and α , the following indeterminate equation is obtained from (7):

$$ls - q\alpha = \beta. \quad (8)$$

Here, since q is a prime number and known, taking both sides modulo q gives

$$ls = \beta \pmod{q} \quad (9)$$

$$s = \frac{\beta}{l} \pmod{q}. \quad (10)$$

Thus, s can be determined.

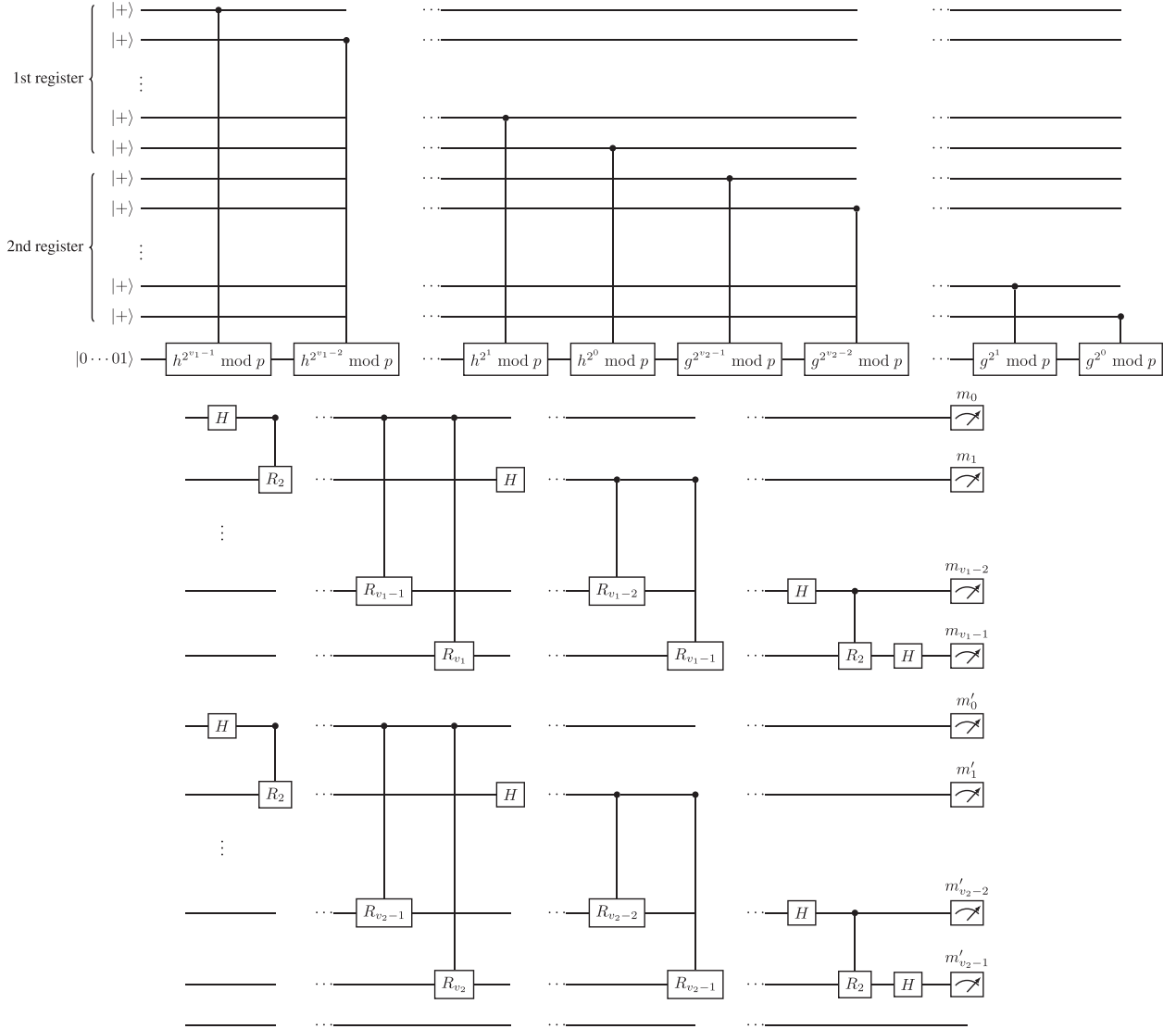


FIGURE 1. Quantum circuit for solving the DLP using Shor's algorithm. Let $R_j = \begin{bmatrix} 1 & 0 \\ 0 & \exp(-2\pi i/2^j) \end{bmatrix}$. The first part of the circuit on the top is for modular exponentiation, and the second part of the circuit on the bottom is for the inverse quantum Fourier transform.

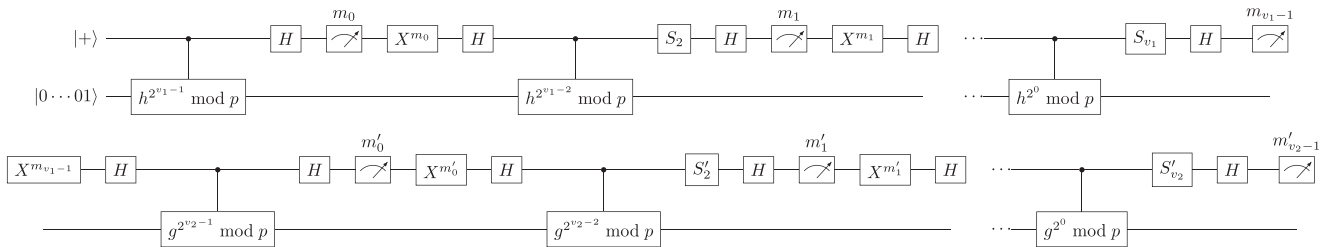


FIGURE 2. Quantum circuit for solving the DLP using Shor's algorithm when the number of qubits in the control registers is set to 1 by employing the semiclassical inverse quantum Fourier transform. Let $S_j = \begin{bmatrix} 1 & 0 \\ 0 & \exp(-2\pi i \sum_{l=2}^j m_{j-l}/2^l) \end{bmatrix}$ and $S'_j = \begin{bmatrix} 1 & 0 \\ 0 & \exp(-2\pi i \sum_{l=2}^j m'_{j-l}/2^l) \end{bmatrix}$.

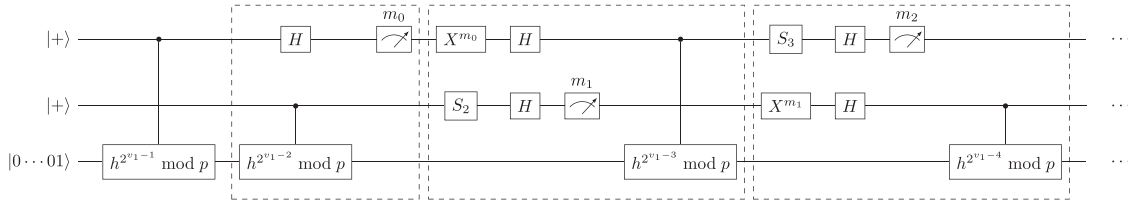


FIGURE 3. Quantum circuit for solving the DLP using Shor's algorithm when the number of qubits in the control registers is set to two by employing the semiclassical inverse quantum Fourier transform and reducing the time depth of the quantum circuit. The parts enclosed by dashed lines can be executed in parallel.

In previous research, continued fraction expansion was used [22], but this rounding method allows for a slightly faster determination.

C. SPACE AND TIME COMPLEXITY

Regarding the space complexity represented by the number of qubits, from the process of determining l and β in the classical computation part, the number of qubits in both the first and second registers is

$$v_1 = v_2 = \lceil \log q \rceil + 1. \quad (11)$$

This result is significantly different from the case of factorization. In Shor's algorithm for factoring a composite number N , the order q , which is the denominator of the value held by the first register after the inverse quantum Fourier transform, is unknown. Therefore, to accurately estimate the true rational number from the measurement result, the first register requires $2\lceil \log N \rceil + 1$ qubits, and the continued fraction expansion algorithm must be executed from there [38]. Consequently, in the case of the DLP, the number of qubits required for the first and second registers is given by (11), and the estimation uses rounding and an indeterminate equation after multiplying by q instead of continued fraction expansion, which is different from factorization. Note that there is a method to add extra qubits to the first register to achieve a success probability of over 99% [22], but this consideration is omitted here due to the exponential load increase on classical computers running quantum simulators as the number of qubits increases.

The third register only needs to store values less than p , so the number of qubits required is

$$w = \lceil \log p \rceil. \quad (12)$$

In addition, let A_{space} be the auxiliary qubits for modular exponentiation. This varies depending on the addition method adopted in modular exponentiation, and the specific form will be discussed in the next section. Therefore, the space complexity is

$$v_1 + v_2 + w + A_{\text{space}} = \lceil \log p \rceil + 2\lceil \log q \rceil + 2 + A_{\text{space}}. \quad (13)$$

However, as shown in the quantum computation part, if the semiclassical inverse quantum Fourier transform is used, the first and second registers can be combined into one or two

qubits, so

$$1 + w + A_{\text{space}} = \lceil \log p \rceil + 1 + A_{\text{space}} \quad (14)$$

or

$$2 + w + A_{\text{space}} = \lceil \log p \rceil + 2 + A_{\text{space}}. \quad (15)$$

Next, we describe the time complexity represented by the number of gates. The modular exponentiation is executed with the first and second registers as control. In factorization, the time complexity of modular exponentiation is expressed in polynomial order of the bit length of the value to be factored by any known method [41]. From this, similarly in the DLP, when the time complexity of modular exponentiation is expressed as A_{time} , it can be written as $A_{\text{time}} = O(\lceil \log p \rceil^m \lceil \log q \rceil^n)$. As with space complexity, the values of m and n for each addition method will be discussed in the next section. Since the time complexity of modular exponentiation is the most dominant in the entire algorithm regardless of the addition method used [10], [41], the overall time complexity is

$$A_{\text{time}} = O(\lceil \log p \rceil^m \lceil \log q \rceil^n). \quad (16)$$

III. CONSTRUCTION OF THE QUANTUM CIRCUIT

Among the quantum circuits of Shor's algorithm, the modular exponentiation, which is the most computationally intensive process, is realized by the Mod-EXP circuit [41]. The number of required qubits and gates varies depending on the implementation method of the adder. There are three types of adder implementations: Q-ADD, GT-ADD, and R-ADD. Q-ADD and GT-ADD require fewer qubits but result in an increased number of gates, whereas R-ADD requires more qubits but fewer gates [41]. On a quantum simulator, as the number of qubits increased, the required memory grows exponentially, making execution difficult. Therefore, for creating quantum circuits to run on a quantum simulator, Q-ADD was used. For resource estimation at the 2048-bit scale, R-ADD, which can reduce the number of gates and shorten execution time, was used.

Regardless of whether the circuit is based on Q-ADD or R-ADD, as in [12], [18], and [41], Mod-EXP consists of Mod-MUL, Mod-PS, Mod-ADD, and ADD.

- 1) Mod-EXP(a): $|x\rangle |1\rangle \rightarrow |x\rangle |a^x \bmod p\rangle$.
- 2) Mod-MUL(d): $|y\rangle \rightarrow |dy \bmod p\rangle$.

- 3) Mod-PS(d): $|y\rangle |t\rangle \rightarrow |y\rangle |t + dy \bmod p\rangle$.
- 4) Mod-ADD(d): $|y\rangle \rightarrow |y + d \bmod p\rangle$.
- 5) ADD(d): $|y\rangle \rightarrow |y + d\rangle$.

Here, we construct quantum circuits for the DLP rather than for factorization, but the construction method itself is mostly common. However, unlike factorization, there are two variables, p and q , so we will explain how A_{space} and A_{time} introduced in the previous section are described for both Q-ADD-based circuits and R-ADD-based circuits.

A. Q-ADD BASED CIRCUITS

Q-ADD is a method that uses the quantum Fourier transform as an adder [27], [42]. The construction method for modular exponentiation circuits using Q-ADD is the same as in [12] and [18]. Therefore, when implementing Mod-ADD, additional bits for the range of the complement of the modulo p and an extra bit are required (see [27, Fig. 5] for details), resulting in

$$A_{\text{space}} = (\lceil \log p \rceil + 1) + 1 = \lceil \log p \rceil + 2. \quad (17)$$

Regarding time complexity, as in [12], [18], and [27], it takes $O(\lceil \log p \rceil^3)$ to execute Mod-MUL once within the range of the modulo p , and since the number of executions is $v_1 + v_2 = 2\lceil \log q \rceil + 2 = O(\lceil \log q \rceil)$, the overall time complexity is given by (16) as

$$O(\lceil \log p \rceil^3 \lceil \log q \rceil). \quad (18)$$

B. R-ADD-BASED CIRCUITS

R-ADD is a ripple carry adder [28]. Previous studies estimating the resources of Shor's algorithm on surface codes also based their addition method on this [10]. As in Q-ADD, the construction method of the modular exponentiation circuit using R-ADD is similar to that in [12], [18], and [41], and in addition to the number of qubits required for Q-ADD, $\lceil \log p \rceil$ qubits are needed, resulting in

$$A_{\text{space}} = (\lceil \log p \rceil + 2) + \lceil \log p \rceil = 2\lceil \log p \rceil + 2. \quad (19)$$

While there are differences in the part related to the number of gates compared to factorization, but they do not affect the order of time complexity, so they are described in the Appendix. Thus, as in [12], [18], and [41], it takes $O(\lceil \log p \rceil^2)$ to execute Mod-MUL once in the range of modulo p , and since this is done $v_1 + v_2 = O(\lceil \log q \rceil)$ times, the overall time complexity is

$$O(\lceil \log p \rceil^2 \lceil \log q \rceil). \quad (20)$$

IV. SIMULATING QUANTUM CIRCUITS

The results of executing Shor's algorithm for solving the DLP on a quantum simulator are presented. When executing the generated circuits, we utilized Q-ADD, which requires fewer qubits as previously mentioned, and employed mpiQulacs 1.3.1 [40], a distributed full-state vector simulator developed by Fujitsu. We used a PRIMEHPC FX700 with one node for up to 28 qubits, four nodes for 29–32 qubits,

and up to 256 nodes for 36 qubits. mpiQulacs is optimized for the high memory bandwidth of the A64FX CPU used in PRIMEHPC FX700 [40]. In addition, after generating the quantum circuits, we applied `optimize_light` from mpiQulacs as an optimization step. This optimization not only includes merging gates using the greedy method from Qulacs [39] but also reduces interprocess communication by fused-swap gates [40].

On the other hand, for circuit generation aimed at resource estimation, we utilized R-ADD, which requires fewer gates. For optimization, we used the standard `optimize_light` from Qulacs [39].

A. RUNNING GENERATED CIRCUITS

We executed quantum circuit simulations using Q-ADD for all 1860 combinations of prime numbers p and q that satisfy

$$q|(p-1) \quad (21)$$

mentioned in the introduction up to 32 qubits. The complete list of these combinations is shown in Table 2. The solution s is chosen uniformly at random from $[0, q)$ for each (p, q) . The simulation results confirmed that the correct solution s was obtained for all (p, q) combinations.

In addition, as an experiment with large numbers, we solved the DLP of a safe-prime group with $(p, q) = (263, 131), (347, 173), (359, 179), (383, 191), (467, 233), (479, 239), (503, 251)$, which can be represented with 36 qubits, using a quantum simulator. Here, p is 9 bits and q is 8 bits. Except for setting $s = 100$ in $(503, 251)$, s is randomly chosen from $[0, q)$ in these combinations. On the other hand, for the Schnorr group, if p and q are chosen such that the time complexities of the NFS and the baby-step giant-step methods match, p becomes less than q at the scale of 36 qubits, causing a breakdown. Therefore, the Schnorr group was not used in this experiment. As a result, the correct solution s was obtained, and the time to simulate the optimized quantum circuit until 10 000 samples were obtained ranged from 30 to 103 min.

For each combination of p and q , 10 000 samples were experimented, and the success probability of correctly obtaining the solution s from the measurement results was calculated according to the procedures of the classical computation part. To describe the method for calculating the success probability more accurately, a quantum circuit generated for one (p, q) pair is executed 10 000 times to obtain 10 000 measurement results. For each of those measurement results, β and l are calculated, and if an s satisfying $h \equiv g^s \bmod p$ is obtained, it is considered a success; if not, it is considered a failure. The number of successes divided by 10 000 gives the success probability for that (p, q) pair. The change in success probability with q as the horizontal axis is shown as a box plot in Fig. 4.

First, let us focus on the minimum and maximum values of the success probability. The minimum success probability was 0.4181, confirmed at $q = 3$, and the maximum was 0.9133, confirmed at $q = 13$. The maximum value appears

TABLE 2. All 1860 Possible Combinations of p and q up to 32 Qubits

p		# q																			#safe-prime group	total	
min	max	2	3	5	7	11	13	17	19	23	29	31	37	41	43	47	53	61	83	89	113		
0	300	62	28	15	9	4	4	3	2	3	2	0	2	1	1	1	1	0	1	1	1	11	141
300	600	47	22	10	7	5	5	3	3	2	2	2	0	0	1	0	0	1	–	–	–	0	251
600	900	45	24	11	9	6	2	2	2	2	0	1	–	–	–	–	–	–	–	–	–	0	355
900	1200	42	20	12	7	4	4	3	0	2	1	0	–	–	–	–	–	–	–	–	–	0	450
1200	1500	43	21	10	6	4	5	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	539
1500	1800	39	22	7	5	2	1	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	615
1800	2100	39	17	10	5	2	5	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	693
2100	2400	40	21	12	8	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	774
2400	2700	36	16	8	8	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	842
2700	3000	37	16	8	6	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	909
3000	3300	33	18	10	5	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	975
3300	3600	40	22	11	7	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1055
3600	3900	36	19	8	6	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1124
3900	4200	35	17	6	2	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1184
4200	4500	36	15	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1235
4500	4800	36	19	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1290
4800	5100	35	17	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1342
5100	5400	31	13	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1386
5400	5700	38	22	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1446
5700	6000	33	15	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1494
6000	6300	36	18	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1548
6300	6600	34	18	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1600
6600	6900	34	18	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1652
6900	7200	32	13	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1697
7200	7500	31	17	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1745
7500	7800	37	19	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1801
7800	8100	31	13	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1845
8100	8400	10	5	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1860

Each row represents the number of values q within the minimum and maximum range of the target p . For example, the number of combinations of p and q where $q = 5$ and $p \in [300, 600]$ is 10. Additionally, – indicates combinations of p and q that cannot be represented within 32 qubits. The #safe-prime group represents the number of combinations of p and q that fall into the safe-prime group within the range of p in that row.

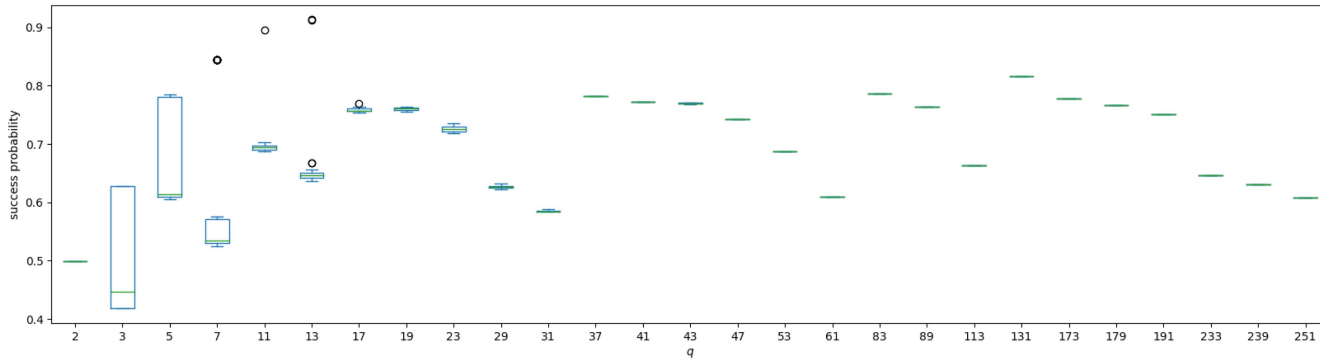


FIGURE 4. Box plot showing the change in success probability with the size of q . The bottom of the box represents the first quartile, the top represents the third quartile, and the line inside represents the median. The lower and upper ends of the whiskers represent the furthest data within $1.5 \times$ the interquartile range from the box. Circles are outliers that are further away than these.

as an outlier in Fig. 4. Such minimum and maximum values have not been found in previous theory [22] or simulations [23]. Here, Fig. 5 shows a heatmap of the success probability for the executed quantum circuit, with $s \in [0, q]$ and for each q . For the same (s, q) , multiple success probabilities may exist due to different p , so we plotted both the minimum and maximum. However, since there was almost no difference between them, only the minimum is shown in Fig. 5, and the maximum is included in the Appendix, available online. Generally, the success probability changes depending on q , as per the heuristic analysis in [22], but as shown in Fig. 5, it deviates from this behavior and results in a high success probability, especially when $s = 0$. As mentioned in [22], one of the effective conditions for heuristic analysis is that s/q does not have a good rational approximation with a small denominator, and it is considered that this is not satisfied when $s = 0$, especially in situations where q

is a prime number. On the other hand, the success probability close to the minimum value is observed to uniquely appear when q is small ($q < 11$) from Fig. 4. In Fig. 5, since the success probability is almost equal for common q except for $s = 0$, it is unlikely that such a small success probability exists for larger q , as seen in the success probability for some s obtained in this simulation. Therefore, the range of q that yields a success probability close to the minimum value has been clarified by this simulation.

Next, let us focus on the behavior of the success probability. As shown in Fig. 4, the success probability changes periodically as q increases, with a range of approximately 60–80%. Specifically, the success probability reaches a local minimum value just before q is a power of 2 ($q = 7, 13, 31, 61$) and then immediately reaches a local maximum value. For $q = 3$, it can also be considered to reach a local minimum in Fig. 5, except for the success probability at

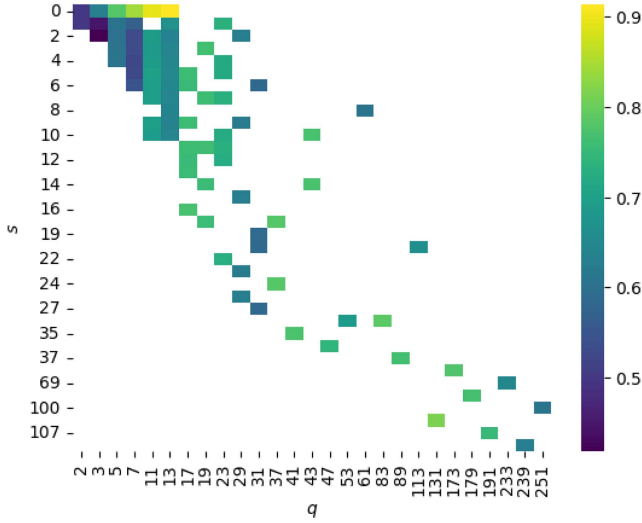


FIGURE 5. Heatmap of success probability with s on the vertical axis and q on the horizontal axis, where s is randomly determined for combinations of (p, q) . Here, only the plot of the minimum success probability for the same (s, q) is shown. Since there was almost no difference between this plot and that showing the maximum success probability, the latter is included in the Appendix, available online.

$s = 0$. Ekerå's heuristic theoretical analysis [22] shows that the success probability changes periodically with q , and that the success probability is approximately 60% as a local minimum value at values of q that are one less than powers of 2, and approximately 82% as a local maximum value at values of q that are one more than powers of 2. This experimental results confirm that heuristic theoretical analysis. Furthermore, we were able to depict a waveform with an asymmetric shape for q , where the value decreases gradually from the local maximum value and then drops sharply as it approaches the local minimum value. Note that [22, Th. 3] shows that when certain conditions listed there, including classical post-processing, are met, the success probability asymptotically approaches 1 as q approaches infinity. This time, since these conditions are not met and it is under the most naive conditions, no significant change in the probability range was observed, except for a slight upward trend in the maximum value within the initial experimental range of q .

B. RESOURCE ESTIMATION BY GENERATED CIRCUITS

We estimate the resources using quantum circuits with R-ADD, which can reduce the number of gates. In [12] and [18], ten composite numbers were randomly selected for each bit length of the values to be factored, and quantum circuits were generated to solve them to obtain resource values. The bit lengths of the composite numbers ranged from 8 to 24, resulting in a total of 170 composite numbers used in the experiments. On the other hand, in the DLP, there are degrees of freedom not only in the modulo p but also in the order q . Therefore, for each bit length of p and q that can be represented by the given number of qubits, five specific values of p and q were

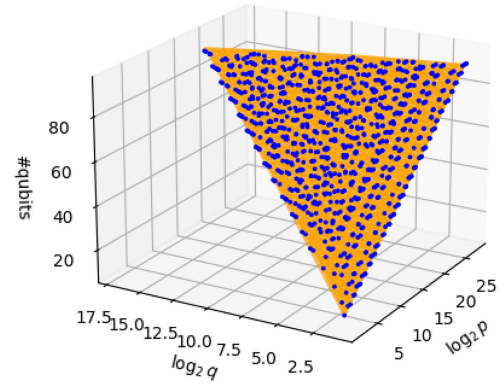


FIGURE 6. Number of qubits. The blue dots represent the results obtained from each experiment, and the orange surface represents (22).

TABLE 3. Average of the Test Data and the Root-Mean-Squared Error (RMSE) From the Test Data Averaged Over All Folds, Obtained by Performing Fitting Using (23) on Each Training Dataset in the Tenfold Cross Validation

	Average	RMSE
Number of gates before optimization	7.66×10^6	4.22×10^5
Number of gates after optimization	2.49×10^6	1.36×10^5
depth before optimization	4.20×10^6	2.25×10^5
depth after optimization	1.89×10^6	1.00×10^5
Number of non-Clifford gates before optimization	3.56×10^6	1.95×10^5
Number of non-Clifford gates after optimization	2.38×10^6	1.31×10^5

randomly selected, and quantum circuits were generated and experimented with. For example, when the number of qubits is 41, the combinations of bit lengths of p and q that can be represented are $(\lceil \log p \rceil, \lceil \log q \rceil) = (9, 5), (11, 2)$ according to (19) and (13). Among them, only $(p, q) = (283, 47), (367, 61), (431, 43)$ exist for the former, so all three combinations were used. For the latter, more than five combinations exist, so $(p, q) = (1871, 5), (1901, 5), (1321, 5), (1597, 7), (1231, 5)$ were randomly selected. The number of qubits ranged from 12 to 90, resulting in a total of 1015 combinations of p and q used in the experiments. These (p, q) satisfy (21), and each s is randomly chosen from $[0, q)$.

First, the number of qubits matches the surface depicted by the continuous form of the equation obtained by substituting (19) into (13) as follows (see Fig. 6) :

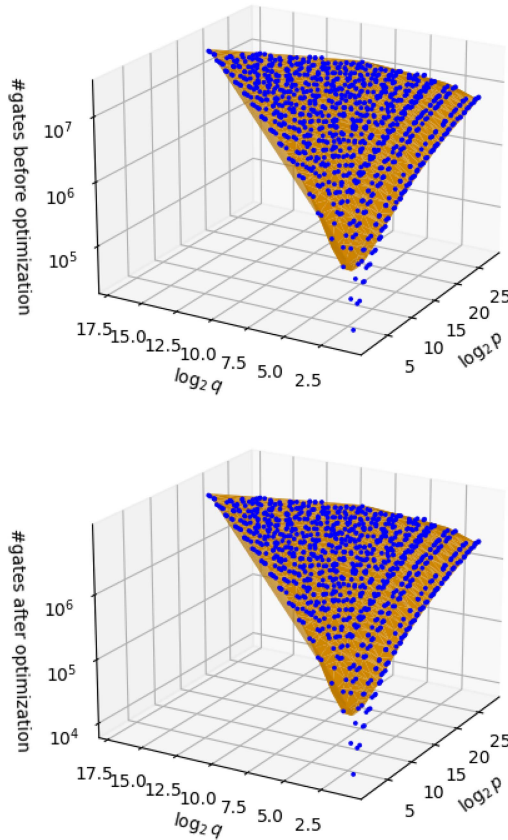
$$3 \log p + 2 \log q + 4. \quad (22)$$

On the other hand, the depth before and after optimization and the number of non-Clifford gates before and after optimization were fitted using the equation based on (20)

$$a(\log p)^2 \log q + b(\log p)^2 + c \log p \log q + d \log p + e \log q + f \quad (23)$$

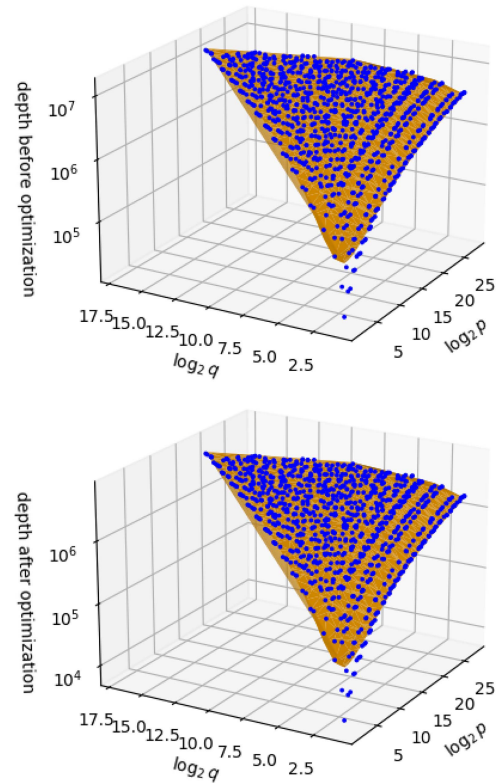
TABLE 4. Parameters of (23) Obtained From the Fitting

	a	b	c	d	e	f
Number of gates before optimization	2843.058	5260.803	28708.363	-62 171.358	-19 2948.809	459 828.645
Number of gates after optimization	923.891	1720.901	9263.616	-19 976.457	-62 110.960	148 050.262
depth before optimization	1534.728	3178.924	14 408.472	-30 183.329	-97 531.979	228 340.308
depth after optimization	694.800	1421.309	6349.034	-13 164.850	-42 942.880	100 263.303
Number of non-Clifford gates before optimization	1320.030	2444.355	13 297.437	-28 879.570	-89 484.421	213 398.262
Number of non-Clifford gates after optimization	882.216	1647.665	8882.046	-19 108.039	-59 544.637	141 771.038

**FIGURE 7.** Number of gates before and after optimization. The blue dots represent the results obtained from each experiment, and the orange surface represents (23) using the parameters shown in Table 4.

and optimized for $a, b, c, d, e, f \in \mathbb{R}$ using the nonlinear least squares method (curve_fit) of SciPy 1.10.1 [43]. To evaluate the accuracy of this fitting, tenfold cross validation was performed, and the average value and mean-squared error of the test data for each fold were averaged across all folds, as shown in Table 3. As shown, the errors are within about one-tenth of the average value. From here, regardless of whether optimization is applied or not, the number of gates, circuit depth, and the number of non-Clifford gates can all be expressed by (23), with the differences being only in the coefficients.

The fitting results using all 1015 combinations as training data are shown in Figs. 7–9. The parameters are shown in Table 4.

**FIGURE 8.** Depth before and after optimization.

Based on these results, we estimated the resources when p is 2048 bits. When p is 2048 bits, the bit length of q differs between safe-prime groups and Schnorr groups. In safe-prime groups, q is approximately 2047 bits to satisfy $p = 2q + 1$. On the other hand, in Schnorr groups, q is 256 bits [44] to match the time complexity of the NFS, which is determined by p , and the baby-step giant-step method, which is determined by q . As shown in Table 5, the required number of gates and depth differ between safe-prime groups and Schnorr groups. In classical computation, as is evident from the way p and q are determined in Schnorr groups, the time complexity required to solve the DLP for both, where q is 256 bits or more, is the same. However, this is not the case in quantum computation because the overall time complexity depends on both p and q , as shown in (16). In the rest of this section, we will explore the differences in computational complexity between safe-prime groups and Schnorr groups

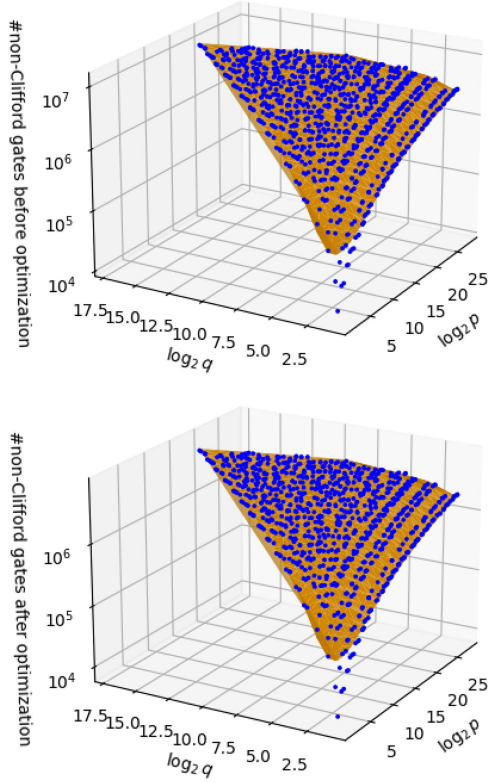


FIGURE 9. Number of non-Clifford gates before and after optimization.

TABLE 5. Resource Estimation Results for Safe-Prime and Schnorr Groups When p Is 2048 Bits, Using R-ADD [28] for Modular Exponentiation

(p , q)	Safe-prime (2048, 2047)	Schnorr (2048, 256)
Number of qubits	10241	6659
Number of gates before optimization	2.46×10^{13}	3.09×10^{12}
Number of gates after optimization	7.98×10^{12}	1.00×10^{12}
depth before optimization	1.33×10^{13}	1.67×10^{12}
depth after optimization	6.00×10^{12}	7.55×10^{11}
Number of non-Clifford gates before optimization	1.14×10^{13}	1.43×10^{12}
Number of non-Clifford gates after optimization	7.62×10^{12}	9.59×10^{11}

in quantum computation from the perspective of computational complexity analysis.

The strength of Schnorr groups under quantum computation can be measured by the size of p in safe-prime groups. Specifically, when using Shor's quantum algorithm, we investigated the size of p in safe-prime groups that matches the computational complexity of the Schnorr group with $p = 2048$ bits. In (23), for safe-prime groups, since $\log q = \log p - 1$, it can be transformed as follows:

$$a(\log p)^2(\log p - 1) + b(\log p)^2 + c \log p(\log p - 1) + d \log p + e(\log p - 1) + f \quad (24)$$

TABLE 6. Size of p Where the Substitution of Each Parameter Into the (25) for the Safe-Prime Group Matches the $p = 2048$ Bit Resource Estimation Result of the Schnorr Group in Table 5

	$ p $
Number of gates before optimization	1024.510
Number of gates after optimization	1024.533
depth before optimization	1024.846
depth after optimization	1024.861
Number of non-Clifford gates before optimization	1024.515
Number of non-Clifford gates after optimization	1024.532

$$= a(\log p)^3 + (-a + b + c)(\log p)^2 + (-c + d + e) \log p + (-e + f). \quad (25)$$

By substituting the parameters (see Table IV) into (25) for each resource and finding p that matches the results of the Schnorr group in Table 5, we obtain Table 6. Here, it is shown that regardless of optimization, the number of gates, circuit depth, or the number of non-Clifford gates, under Shor's quantum algorithm, a Schnorr group with $p = 2048$ bits has almost the same cryptographic strength as a safe-prime group with $p = 1024$ bits. Therefore, it can be inferred that there is a universality independent of the parameters a, b, c, d, e , and f in (23).

Henceforth, we theoretically analyze a more general relational expression for p where the cryptographic strength of the Schnorr group and the safe-prime group becomes equal in quantum computation. According to [2], the combination of bit lengths for p and q in Schnorr groups is $(|p|, |q|) = (1024, 160), (2048, 224), (2048, 256), (3072, 256)$, so approximately

$$\log q = \chi \log p \quad (26)$$

where

$$\chi \approx 1/10 \quad (27)$$

is expressed. Then, from (23)

$$\begin{aligned} & a(\log p)^2(\chi \log p) + b(\log p)^2 + c(\log p)(\chi \log p) \\ & + d \log p + e \chi \log p + f \\ & = a\chi(\log p)^3 + (b + c\chi)(\log p)^2 + (d + e\chi) \log p + f \end{aligned} \quad (28)$$

is obtained. Let p_{sp} be p of the safe-prime group and p_s be p of the Schnorr group. By transforming the equation that connects (25) and (29) with an equal sign, we obtain

$$\begin{aligned} & a((\log p_{sp})^3 - \chi(\log p_s)^3) + (-a + b + c)(\log p_{sp})^2 \\ & - (b + c\chi)(\log p_s)^2 + (-c + d + e) \log p_{sp} \\ & - (d + e\chi) \log p_s - e = 0. \end{aligned} \quad (30)$$

By ignoring the second-order terms of $\log p_{sp}$ and $\log p_s$, we have

$$(\log p_{sp})^3 \sim \chi(\log p_s)^3 \quad (31)$$

$$\log p_{\text{sp}} \sim \sqrt[3]{\chi} \log p_{\text{s}}. \quad (32)$$

From (27), it is understood that there is a difference of about twice in the bit length of both. In particular, in the Schnorr group with $(|p|, |q|) = (2048, 256)$, $\chi = 1/8 = 1/2^3$, so $\log p_{\text{sp}} \sim (1/2) \log p_{\text{s}}$. This means that under Shor's algorithm, the cryptographic strength of a Schnorr group with $p = 2048$ bits is almost equivalent to that of a safe-prime with $p = 1024$ bits, reproducing the results shown in Table 6. Therefore, it was shown that the cryptographic strength of the Schnorr group is reduced to about half in terms of the bit length of p in the safe-prime group, regardless of specific parameters or extrapolation accuracy.

V. CONCLUSION

In this article, we conducted large-scale quantum simulations of comprehensive combinations of modulo p and order q for Shor's algorithm solving the DLP, which had not been done before. Through 1860 experiments with all combinations of p and q up to 32 qubits, we obtained a list of actual success probabilities. From this, the correctness of Ekerå's heuristic theoretical analysis [22], which includes periodic fluctuations in success probability due to order q , was demonstrated, and the minimum and maximum values as well as the detailed waveform shape were obtained. In addition, by extrapolating the circuit scale obtained from experiments with larger combinations of p and q , we estimated the necessary resource values for 2048 bits. From this, we showed that under Shor's quantum algorithm, the Schnorr group has weaker cryptographic strength than the safe-prime group, and furthermore, the cryptographic strength of the Schnorr group experimentally and theoretically decreases to about half the size of p for the safe-prime group.

In the future, we aim to extend this to scenarios assuming quantum error correction as in [10] and [11] and to the elliptic curve DLP. In the former, the time to solve the DLP is considered based on the cost of quantum error correction, and it can be predicted from our results on the number of non-Clifford gates, which significantly affects this cost, that the time will be the cryptographic strength at half the p of the safe-prime group in the Schnorr group. In the latter, since various elliptic curves exist, it will be necessary to verify how their cryptographic strength is evaluated under quantum computation.

ACKNOWLEDGMENT

The authors are deeply grateful to Martin Ekerå for his various valuable comments, including existing research on the oscillation of success probability.

REFERENCES

- [1] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Trans. Inf. Theory*, vol. 22, no. 6, pp. 644–654, Nov. 1976, doi: [10.1109/TIT.1976.1055638](#).
- [2] *Digital Signature Standard (DSS)*, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Federal Information Processing Standard 186-4, 2013, doi: [10.6028/NIST.FIPS.186-4](#).
- [3] O. Schirokauer, "Discrete logarithms and local units," *Philos. Trans. Roy. Soc. London Ser. A: Phys. Eng. Sci.*, vol. 345, pp. 409–423, 1993, doi: [10.1098/rsta.1993.0139](#).
- [4] E. Barker, L. Chen, A. Roginsky, A. Vassilev, and R. Davis, *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography*, U.S. Dept. Commerce, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST SP 800-56A Rev. 3, 2018, doi: [10.6028/NIST.SP.800-56Ar3](#).
- [5] S. Pohlig and M. Hellman, "An improved algorithm for computing logarithms over $\text{GF}(p)$ and its cryptographic significance (corresp.)," *IEEE Trans. Inf. Theory*, vol. IT-24, no. 1, pp. 106–110, Jan. 1978, doi: [10.1109/TIT.1978.1055817](#).
- [6] D. Shanks, "Class number, a theory of factorization, and genera," *Proc. Symp. Math. Soc.*, vol. 20, pp. 415–440, 1971, doi: [10.1090/pspum/020/0316385](#).
- [7] *Digital Signature Standard (DSS)*, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Federal Information Processing Standard 186-5, 2023, doi: [10.6028/NIST.FIPS.186-5](#).
- [8] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, 1994, pp. 124–134, doi: [10.1109/SFCS.1994.365700](#).
- [9] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, pp. 1484–1509, 1995, doi: [10.1137/S0036144598347011](#).
- [10] C. Gidney and M. Ekerå, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, Apr. 2021, Art. no. 433, doi: [10.22331/q-2021-04-15-433](#).
- [11] E. Gouzien and N. Sangouard, "Factoring 2048-bit RSA integers in 177 days with 13 436 qubits and a multimode memory," *Phys. Rev. Lett.*, vol. 127, Sep. 2021, Art. no. 140503, doi: [10.1103/PhysRevLett.127.140503](#).
- [12] J. Yamaguchi, M. Yamazaki, A. Tabuchi, T. Honda, T. Izu, and N. Kunihiro, "Estimation of Shor's circuit for 2048-bit integers based on quantum simulator," *Cryptol. ePrint Arch.*, 2023, Art. no. 2023/092. [Online]. Available: <https://ia.cr/2023/092>
- [13] Y. Akahoshi, K. Maruyama, H. Oshima, S. Sato, and K. Fujii, "Partially fault-tolerant quantum computing architecture with error-corrected Clifford gates and space-time efficient analog rotations," *PRX Quantum*, vol. 5, Mar. 2024, Art. no. 010337, doi: [10.1103/PRXQuantum.5.010337](#).
- [14] R. Toshio, Y. Akahoshi, J. Fujisaki, H. Oshima, S. Sato, and K. Fujii, "Practical quantum advantage on partially fault-tolerant quantum computer," *Phys. Rev. X*, vol. 15, 2025, doi: [10.1103/PhysRevX.15.021057](#).
- [15] Y. Akahoshi, R. Toshio, J. Fujisaki, H. Oshima, S. Sato, and K. Fujii, "Compilation of trotter-based time evolution for partially fault-tolerant quantum computing architecture," 2024, *arXiv:2408.14929*, doi: [10.48550/arXiv.2408.14929](#).
- [16] T. Itogawa, Y. Takada, Y. Hirano, and K. Fujii, "Efficient magic state distillation by zero-level distillation," *PRX Quantum*, vol. 6, 2025, Art. no. 020356, doi: [10.1103/thxx-njr6](#).
- [17] C. Gidney, N. Shutty, and C. Jones, "Magic state cultivation: Growing T states as cheap as CNOT gates," 2024, *arXiv:2409.17595*, doi: [10.48550/arXiv.2409.17595](#).
- [18] J. Yamaguchi, M. Yamazaki, A. Tabuchi, T. Honda, T. Izu, and N. Kunihiro, "Experiments and resource analysis of Shor's factorization using a quantum simulator," in *Proc. Int. Conf. Intell. Comput., Smart Commun. Netw. Technol.*, 2024, pp. 119–139, doi: [10.1007/978-981-97-1235-9_7](#).
- [19] M. Ekerå, "Quantum algorithms for computing general discrete logarithms and orders with tradeoffs," *J. Math. Cryptol.*, vol. 15, no. 1, pp. 359–407, 2021, doi: [10.1515/jmc-2020-0006](#).
- [20] M. Ekerå and J. Hästad, "Quantum algorithms for computing short discrete logarithms and factoring RSA integers," in *Post-Quantum Cryptography*, T. Lange and T. Takagi, Eds., Cham, Switzerland: Springer, 2017, pp. 347–363, doi: [10.1007/978-3-319-59879-6_20](#).
- [21] M. Hhan, T. Yamakawa, and A. Yun, "Quantum complexity for discrete logarithms and related problems," in *Proc. Int. Cryptol. Conf.*, 2024, pp. 3–36, doi: [10.1007/978-3-031-68391-6_1](#).
- [22] M. Ekerå, "Revisiting Shor's quantum algorithm for computing general discrete logarithms," 2019, *arXiv:1905.09084*, doi: [10.48550/arXiv.1905.09084](#).

- [23] A. Mandl and U. Egly, "Implementations for Shor's algorithm for the DLP," in *Proc. Jahrestagung der Gesellschaft Informatik*, 2022, pp. 1133–1143, doi: [10.18420/inf2022_96](https://doi.org/10.18420/inf2022_96).
- [24] Y. Aono et al., "The present and future of discrete logarithm problems on noisy quantum computers," *IEEE Trans. Quantum Eng.*, vol. 3, 2022, Art. no. 3102021, doi: [10.1109/TQE.2022.3183385](https://doi.org/10.1109/TQE.2022.3183385).
- [25] O. Regev, "An efficient quantum factoring algorithm," *J. ACM*, vol. 72, 2023, Art. no. 10, doi: [10.1145/3708471](https://doi.org/10.1145/3708471).
- [26] M. Ekerå and J. Gärtner, "Extending Regev's factoring algorithm to compute discrete logarithms," in *Post-Quantum Cryptography*, M.-J. Saarinen and D. Smith-Tone, Eds., Cham, Switzerland: Springer, 2024, pp. 211–242, doi: [10.1007/978-3-031-62746-0_10](https://doi.org/10.1007/978-3-031-62746-0_10).
- [27] S. Beauregard, "Circuit for Shor's algorithm using $2n+3$ qubits," *Quantum Inf. Comput.*, vol. 3, pp. 175–185, Mar. 2003, doi: [10.5555/2011517.2011525](https://doi.org/10.5555/2011517.2011525).
- [28] V. Vedral, A. Barenco, and A. Ekert, "Quantum networks for elementary arithmetic operations," *Phys. Rev. A*, vol. 54, pp. 147–153, Aug. 1996, doi: [10.1103/PhysRevA.54.147](https://doi.org/10.1103/PhysRevA.54.147).
- [29] R. Rivest and R. Silverman, "Are 'strong' primes needed for RSA," *Cryptol. ePrint Arch.*, 2001, Art. no. 2001/007. [Online]. Available: <https://ia.cr/2001/007>
- [30] A. Joux and R. Lercier, "Discrete logarithms in $GF(p)$ —130 digits," *An Email Announcement*, Jun. 2005. [Online]. Available: <https://listserv.nodak.edu/cgi-bin/wa.exe?A2=NMBRTHRY;cc056f17.0506&S=>
- [31] A. Dorofeev, D. Dygin, and D. Matyukhin, "Discrete logarithms in $GF(p)$ —135 digits," *An Email Announcement*, Dec. 2006. [Online]. Available: <https://listserv.nodak.edu/cgi-bin/wa.exe?A2=NMBRTHRY;f8ddbc63.0612&S=>
- [32] T. Kleinjung, "Discrete logarithms in $GF(p)$ —160 digits," *An Email Announcement*, Feb. 2007. [Online]. Available: <https://listserv.nodak.edu/cgi-bin/wa.exe?A2=NMBRTHRY;1c737cf8.0702&S=>
- [33] C. Bouvier, P. Gaudry, L. Imbert, H. Jeljeli, and E. Thomé, "Discrete logarithms in $GF(p)$ —180 digits," *An Email Announcement*, Jun. 2014. [Online]. Available: <https://listserv.nodak.edu/cgi-bin/wa.exe?A2=NMBRTHRY;615d922a.1406&S=>
- [34] T. Kleinjung, C. Diem, A. K. Lenstra, C. Priplata, and C. Stahlke, "Discrete logarithms in $GF(p)$ —768 bits," *An Email Announcement*, Jun. 2016. [Online]. Available: <https://listserv.nodak.edu/cgi-bin/wa.exe?A2=NMBRTHRY;a0c66b63.1606&S=>
- [35] F. Boudot, P. Gaudry, A. Guillevis, N. Heninger, E. Thomé, and P. Zimmermann, "795-bit factoring and discrete logarithms," *An Email Announcement*, Dec. 2019. [Online]. Available: <https://listserv.nodak.edu/cgi-bin/wa.exe?A2=NMBRTHRY;fd743373.1912&S=>
- [36] F. Boudot, P. Gaudry, A. Guillevis, N. Heninger, E. Thomé, and P. Zimmermann, "Comparing the difficulty of factorization and discrete logarithm: A 240-digit experiment," in *Proc. Adv. Cryptol.—CRYPTO 2020: 40th Annu. Int. Cryptol. Conf.*, Santa Barbara, CA, USA, Aug. 2020, doi: [10.1007/978-3-030-56880-1_3](https://doi.org/10.1007/978-3-030-56880-1_3).
- [37] F. Boudot, P. Gaudry, A. Guillevis, N. Heninger, E. Thomé, and P. Zimmermann, "Comparing the difficulty of factorization and discrete logarithm: A 240-digit experiment," in *Proc. 40th Annu. Int. Cryptol. Conf.*, 2020, pp. 62–91, doi: [10.1007/978-3-030-56880-1_3](https://doi.org/10.1007/978-3-030-56880-1_3).
- [38] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*, 10th ed. New York, NY, USA: Cambridge Univ. Press, 2011, doi: [10.1017/CBO9780511976667](https://doi.org/10.1017/CBO9780511976667).
- [39] Y. Suzuki et al., "Qulacs: A fast and versatile quantum circuit simulator for research purpose," *Quantum*, vol. 5, Oct. 2021, Art. no. 559, doi: [10.22331/q-2021-10-06-559](https://doi.org/10.22331/q-2021-10-06-559).
- [40] S. Imamura et al., "mpiQulacs: A distributed quantum computer simulator for A64FX-based cluster systems," 2022, *arXiv:2203.16044*, doi: [10.48550/arXiv.2203.16044](https://doi.org/10.48550/arXiv.2203.16044).
- [41] N. Kunihiro, "Exact analyses of computational time for factoring in quantum computers," *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, vol. 88-A, pp. 105–111, 2005, doi: [10.1093/ietfec/e88-a.1.105](https://doi.org/10.1093/ietfec/e88-a.1.105).
- [42] T. G. Draper, "Addition on a quantum computer," 2000, *arXiv:quant-ph/0008033*, doi: [10.48550/arXiv.quant-ph/0008033](https://doi.org/10.48550/arXiv.quant-ph/0008033).
- [43] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [44] *Digital Signature Standard (DSS)*, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Federal Information Processing Standard 186-3, 2009. [Online]. Available: <https://csrc.nist.gov/pubs/fips/186-3/final>